FULL TECHNICAL ARCHITECTURE & DEPLOYMENT REPORT

Ore Beneficiation Digital Twin — v5

Gravity + WHIMS Iron-Ore Beneficiation Plant Simulator, Hybrid AI Layer,

3-D Live Plant Model, Supervisory MPC & Contractual Acceptance Suite

Model Version: OBDT-2026.10.5 | Reference Plants: IX-OB-01, IX-OB-02, IX-OB-03 (Industry-X)

Engineered by Extractmet Private Limited (IIT Madras Research Park / Centre for Pyrometallurgy)

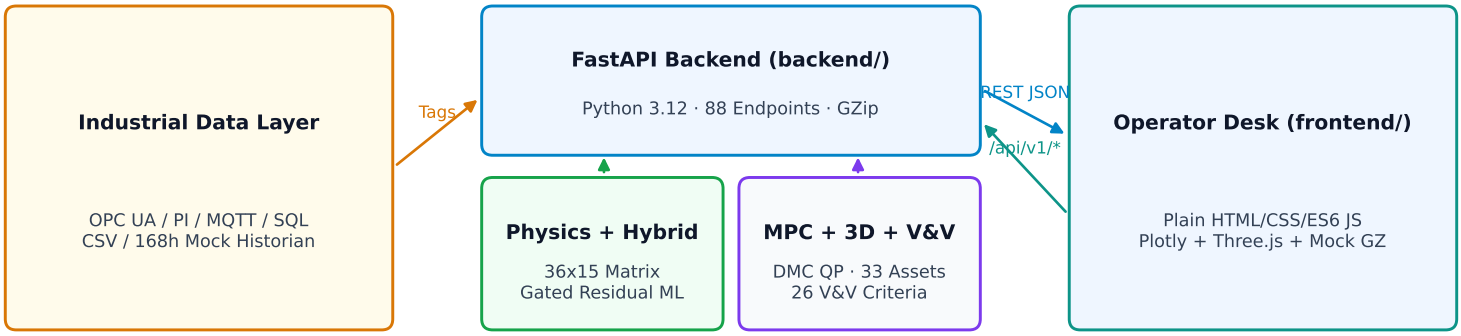
88 Routes FastAPI REST v1	**16 Pages** ES6 + Three.js Desk	**36 x 15 Grid** Size x Liberation PSD	**~155 ms** Full Circuit Solve
33 Assets 5-Level 3D Scene	**6 Modes** Gated Hybrid AI	**31 Limits** 5-Tier MPC / Opt	**26 Criteria** G0-G5 V&V Suite

1. Executive Summary — What This Project Is & How It Works

Ore Beneficiation Digital Twin v5 is a self-contained, industrial-grade process digital twin for an iron-ore washing, gravity concentration, and wet high-intensity magnetic separation (WHIMS) plant. From a single clay-bearing run-of-mine (ROM) hematite/goethite ore stream, the plant simultaneously produces three commercial products: Direct-Reduction / Blast-Furnace Lump Ore (+10 mm), Sinter Fines (-10 mm gravity concentrate), and Pellet-Plant Feed (ultrafine magnetic concentrate). Because tightening grade on one separator shifts mass and impurities (especially Al2O3 clay penalties) into downstream circuits, operators face a coupled three-product metallurgical and economic trade-off.

Version 5 extends the core first-principles simulator, optimizer, ML soft-sensor suite, and supervisory Dynamic Matrix Controller (MPC) with three major capabilities: (1) a data-driven Three.js 3-D plant model bound to live equipment duty and CAD/BIM tag registration, (2) an auditable Gated-Residual Hybrid Mechanistic + Data-Driven layer that guarantees exact fallback to physics outside the training envelope, and (3) a contractual Verification & Validation (V&V) framework enforcing laboratory repeatability limits across Gates G0 through G5.

2. Two-Tier Decoupled Architecture



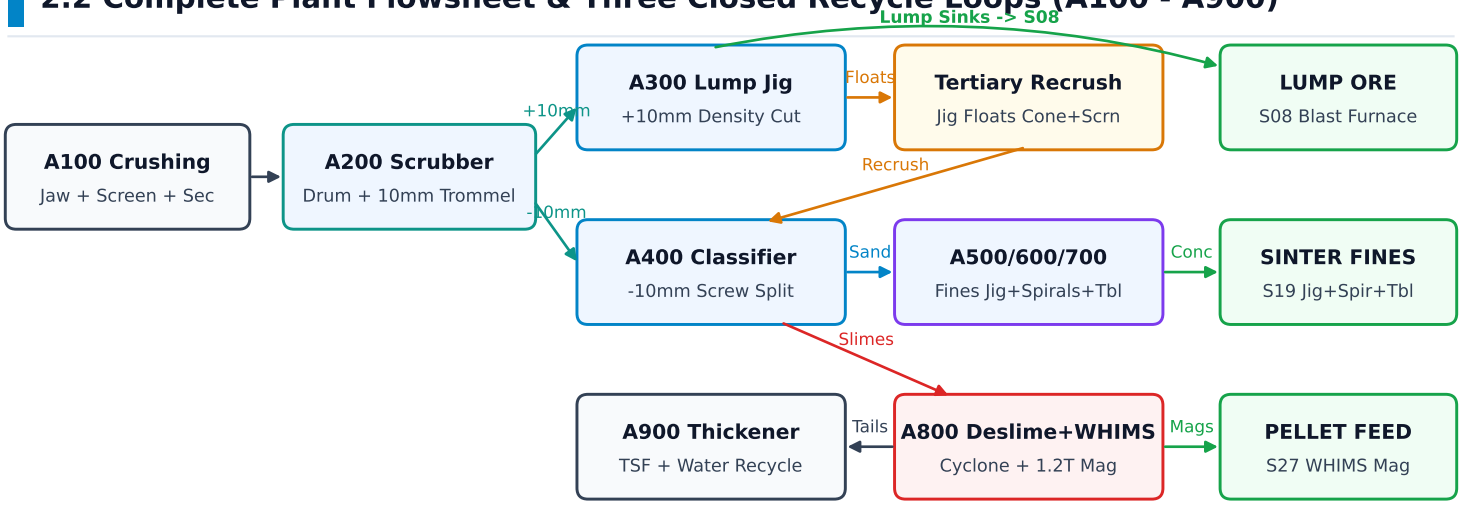
First-Principles Mineral Processing Engine

2-D Particle State Matrix (backend/app/twin/psd.py) & Steady-State Flowsheet Solver (circuit.py)

2.1 The 36 x 15 Size-and-Liberation Particle Matrix

- **36 Geometric Size Classes (1.4 um to 250 mm)**
  Follows a sqrt(2) sieve progression initialized with a Rosin-Rammler distribution from F80 and ore friability. Wide enough to track coarse 250 mm ROM lumps, 10 mm jig beds, 150 um spiral feeds, and <10 um slimes simultaneously.
- **15 Liberation Composition Bins (g = 0.0 to 1.0)**
  Tracks iron-mineral mass fraction from pure gangue (quartz SG 2.65, clay SG 2.55) to free iron oxide (hematite SG 5.15, goethite SG 4.10), bounded by MAX_LIBERATION = 0.90 so unliberated composite middlings remain realistic.
- **Direct Physical Separation Properties**
  Harmonic volume mixing maps every bin g directly to particle Specific Gravity (Taggart concentration criterion = 2.50 in water) and paramagnetic susceptibility chi. Jigs, spirals, tables, and WHIMS separate directly on this state matrix.

2.2 Complete Plant Flowsheet & Three Closed Recycle Loops (A100 - A900)



2.3 Unit-Operation Physics & Wegstein Convergence

- **Crushing & Screening (comminution.py)**
  Primary jaw, closed-loop secondary cone, and tertiary recrush cone modeled with CSS product curves, t10/Axb energy-breakage relationships, Bond Work Index motor power, and Karra/partition vibrating screen efficiency.
- **Rotary Attrition Scrubber (gravity.py)**
  First-order residence-time kinetics 1 - exp(-k*t) driven by % critical drum speed, lifters, and dilution water detach surface clay/ochre coatings from coarse lumps and transfer free Al2O3 gangue into <20 um slimes.
- **Jigs, Spirals & Shaking Tables (gravity.py)**
  Modeled via generalized Tromp partition curves 1 / (1 + exp(-ln(3)*(SG - SG50)/Ep(d))) combined with size-efficiency windows: Lump Jig (+10 mm), Fines Jig (+2 mm), Spirals (75 um - 2 mm), and Shaking Tables on spiral middlings.
- **WHIMS Paramagnetic Capture (separation.py)**
  Balances magnetic traction (~ B^1.75 * matrix_gain / gap^0.45 across 0.3-1.8 T) against slurry drag and coarse gravity detachment, plus wash-water suppression of fine gangue mechanical entrainment.
- **Wegstein Acceleration (_Accel in circuit.py)**
  Extrapolates all 3 circulating loads (secondary screen oversize, tertiary jig-floats recrush, table middlings) over early passes and settles with pure substitution, achieving <0.01% solids closure in ~140-185 ms.

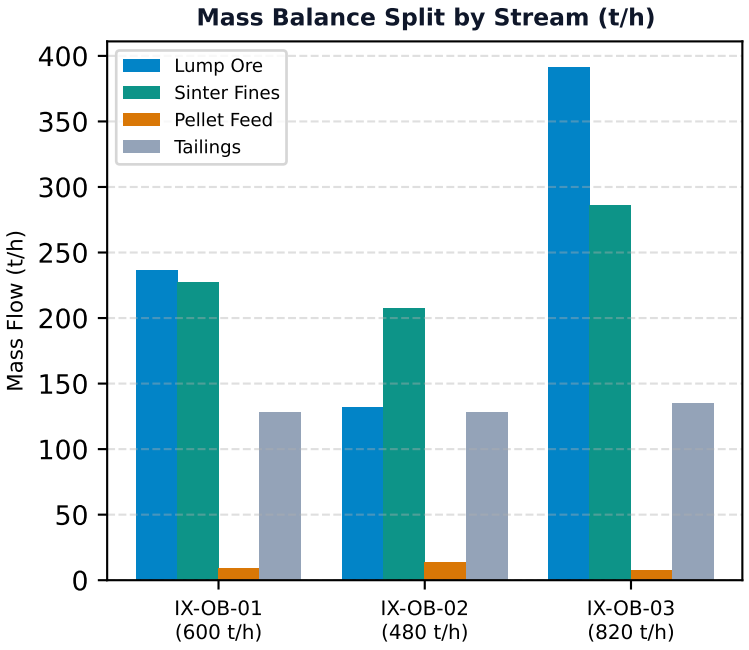
Reference Plant Benchmarks & Economic Trade-Offs

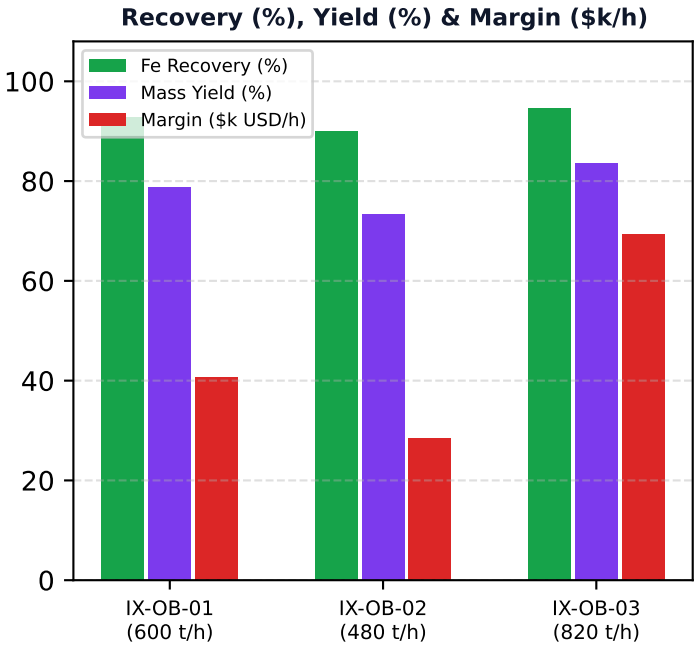
Live Steady-State Engine Outputs Across IX-OB-01, IX-OB-02, and IX-OB-03

3.1 The Three Reference Plants (backend/app/services/plant_service.py)

Plant	Ore Archetype	ROM	Lump t/h (Assay)	Fines t/h	Pellet t/h	Fe Rec	Margin
IX-OB-01	Hematite/Goethite	600	236.3 (63.1% Fe, 2.00% Al)	227.0 (65.2% Fe)	8.7 (60.2% Fe)	92.87%	$40,766/h
IX-OB-02	High-Clay (7.2% Al)	480	131.6 (62.5% Fe, 2.05% Al)	207.4 (64.2% Fe)	13.3 (61.3% Fe)	89.89%	$28,487/h
IX-OB-03	Hard Banded Ore	820	391.5 (65.1% Fe, 1.18% Al)	285.8 (65.9% Fe)	7.6 (60.1% Fe)	94.67%	$69,278/h

3.2 Visual Product Split & Financial Performance (Computed from circuit.solve)





3.3 Commercial Net-Back Pricing, Water Balance & Energy Accounting

- **Net-Back Smelter / Contract Pricing Formula**
  Realised USD/dmt per product = Index + Product Premium (Lump or Pellet) + Fe Differential ((Fe - 62%) * bonus) - Silica Penalty (>4% SiO2) - Alumina Penalty (>free allowance Al2O3) - LOI Penalty (>3%) - Freight.
- **Why Alumina (Al2O3) Is the Binding Constraint**
  Indian goethitic/lateritic ores carry high ochreous clay. Pushing lump yield by lowering Lump Jig cut density pulls composite middlings into Lump Ore, breaching the 2.5% Al2O3 penalty cliff unless Scrubber speed/water are raised.
- **Boundary Water & Power Balance**
  Tracks circulating pump demand vs true boundary losses (product moisture 4%/10%/12%, thickener underflow locked water to TSF, and 1.2% evaporation) minus pond reclaim to compute licensed Fresh Make-up Water (m3/h) and CO2 intensity.

v5 Layer 1 — Hybrid Mechanistic + Data-Driven Architecture

Explicit Composition Modes (composition.py) & 3-Way Domain-of-Validity Gating (domain.py)

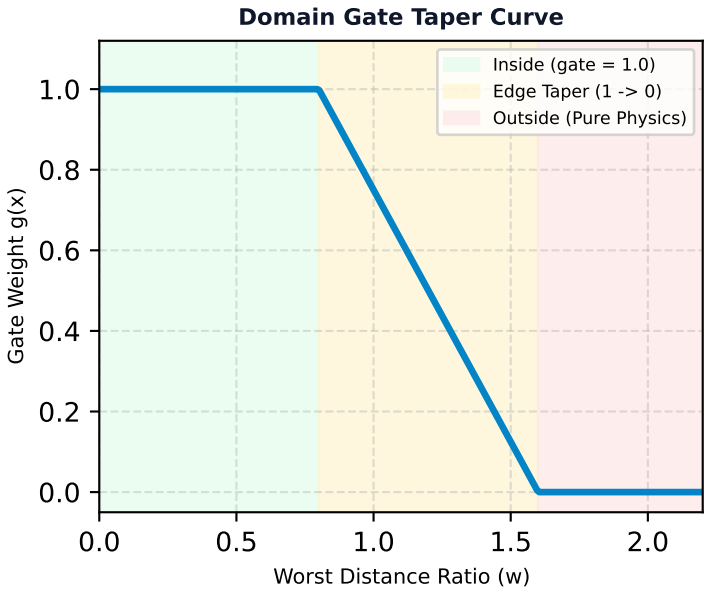
4.1 Six Auditable Hybrid Compositions Implemented & Benchmarked

Mode ID	ML?	Mathematical Formulation	Operational Trade-Off
mechanistic_only	No	First-principles physics solver alone	Extrapolates everywhere; carries site bias
data_only	Yes	Pure Gradient Boosting on plant history	Accurate in-domain; unsafe outside training box
parallel_residual	Yes	Physics + unbounded ML residual correction	Removes bias; can diverge on unseen ore/setpoints
serial_parameter	Yes	ML estimates physics parameter; twin re-solves	100% physically consistent; slowest per call
gated_residual *	Yes	Bounded residual tapered to 0 outside domain	DEFAULT IN SERVICE: accurate + fails safe to physics
ensemble_weighted	Yes	50/50 fixed blend of physics and data-only	Simple baseline; dominated by gated_residual

4.2 How Gated Residual Fails Safe (DomainOfValidity + Clamping)

For all 4 hybrid targets (lump_al2o3_pct, lump_fe_pct, fines_fe_pct, fe_recovery_pct), Gated Residual computes: y_hybrid = y_physics + gate(x) * clamp(r_ML(x)). First, clamp() bounds the raw ML correction to min(3 * sigma_residual, 25% * |y_physics|). Second, DomainOfValidity.score(x) runs three independent out-of-distribution checks and takes the worst ratio w = max(box_ratio, mahalanobis_ratio, nearest_neighbour_ratio):

- **1. Hyperbox Check (box_ratio)**
  Flags any feature exceeding its [min, max] training range.
- **2. Mahalanobis Ratio (combination)**
  95th-percentile covariance check catches unseen combinations of individually normal set-points.
- **3. Nearest-Neighbour Ratio (local density)**
  Catches sparse interior holes inside the convex hull.
- **Explicit Operator Attribution**
  Every prediction reports exact % from physics vs % from ML, plus 95% confidence intervals.



Contractual Safety Guarantee (Verified Automatically by Extrapolation Walk)

- Inside training envelope (w <= 0.80): full bounded ML residual correction removes plant-specific bias.
- Envelope boundary (0.80 < w < 1.60): linear taper smoothly blends ML residual down toward zero.
- Outside training domain (w >= 1.60): gate === 0.0 -> Hybrid output equals First-Principles Twin EXACTLY.
- Tested in CI & Validation Gate G1 via `/api/v1/hybrid/extrapolation` across extreme ROM feed & clay sweeps.

Supervisory MPC, Constrained Optimisation, ML & 3-D Plant Twin

Control Layer (app/control/), Soft Sensors & ML (app/ml/), and Tagged 3-D Scene Graph (app/plant3d/)

5.1 Supervisory Dynamic Matrix Control (MPC) & RGA Advisory

- **Automated Step-Response Identification (identification.py)**
  Perturbs Manipulated Variables (MVs: ROM feed, scrubber speed/water, lump jig SG/hutch water, spiral splitters, WHIMS Tesla) on the twin to fit First-Order Plus Dead-Time (FOPDT: gain K, tau, theta) models and Relative Gain Array (RGA) pairings.
- **Condensed DMC Quadratic Program (mpc.py)**
  Every 5-min control cycle (P=24 prediction horizon, M=4 move horizon), solves a scaled SLSQP Quadratic Program minimizing zone-tracking error + MV move suppression + priority-weighted soft zone slack, with DMC measurement bias feedback.
- **Advisory-Only DCS Safety Boundary**
  Generates a structured DCS Handover payload (`/api/v1/mpc/handover`) checking data health and operator persona permissions; set-point changes always require operator confirmation in the plant DCS.

5.2 Five-Tier Constraint Hierarchy (31 Limits) & 8 Objectives

Priority Tier	Category	Key Monitored Plant Constraints (constraints.py)
Tier 1 (Wt 400)	Safety & Equipment	Crusher/scrubber kW <=90-92%, screen <=26 t/h/m2, thickener rake torque <=80%
Tier 2 (Wt 120)	Product Contract	Lump Fe >=61.5%, Lump Al2O3 <=2.5%, Fines Fe >=63.0%, Lump yield >=28%
Tier 3 (Wt 60)	Environmental Licence	Fresh water <=1500 m3/h, recycle TSS <=180 mg/L, tails U/F >=48% solids
Tier 4 (Wt 25)	Process Stability	Spiral/table/WHIMS loading <=1.0x, jig looseness [0.7-1.6], cyclone kPa bands
Tier 5 (Wt 8)	Business Efficiency	Overall Fe recovery >=88%, mass yield >=72%, energy <=3.2 kWh/t, clay rem >=40%

5.3 Tagged 3-D Plant Model & Site CAD Registration (app/plant3d/)

- **Zero Hardcoded Layout in Browser (33 Assets, 30 Routes, 5 Floors)**
  Backend sends a complete 3-D scene graph (+X east, +Y elevation L00=0m to L04=34m, +Z north) across 19 geometric archetypes (jaw/cone crushers, trommel, jig banks, spirals, WHIMS carousel, thickener) with automated 3-D bounding-box clash detection.
- **Live Twin Overlays & Interactive Inspection (plantmodel.js)**
  Three.js colors meshes in real time by constraint status, duty %, motor power, throughput, Fe grade, or slurry density, with floor clipping planes and click-to-inspect asset datasheets and connected streams.
- **CAD / BIM Ingestion & Umeyama Registration (importers.py)**
  Supports glTF/GLB natively plus IFC, AVEVA RVM, STEP, and E57/LAS point clouds. `match_nodes` enforces strict 3-way tag matching (matched / ambiguous / unmatched — never guesses!), and `solve_registration` computes closed-form SVD Umeyama 3-D similarity transforms from >=3 survey control points.

v5 Layer 3 — Verification, Validation & Contractual Acceptance

Statistical Metrics, Tolerance Classes A/B/C & 6-Gate Acceptance Protocol (app/validation/)

6.1 Three Contractual Engineering Rules Enforced in Code

Why Standard ML Metrics Fail in Metallurgical Plant Contracts — And How OBDT v5 Fixes Them

1. Repeatability Floor: No tolerance may be tighter than the lab/instrument 1-sigma repeatability (unit-tested!).
2. Assay vs Tonnage Gating: Assays gate on ABSOLUTE error (% Fe / % Al2O3); flows gate on RELATIVE error (MAPE %).
3. Non-Discriminating Noise Rule: If a KPI's test range sits inside measurement noise, R2 is shown but excluded from gating.

6.2 14 KPI Criteria + 12 System Criteria Across Classes A, B, and C

Tolerance Class	Acceptance Gate Rule	Covered KPIs & Threshold Examples
Class A (1.5x sigma)	Contractual — ALL must pass	Lump Fe (<=0.60%), Lump Al2O3 (<=0.16%), Fines Fe (<=0.60%), Rec, Yield
Class B (2.0x sigma)	Operational — Pass/concession	Pellet Fe, ROM/Lump/Fines t/h (MAPE <=2.5-3.0%), Tails Fe, Power, Water
Class C (3.0x sigma)	Indicative — Reported trend	Secondary diagnostic splits & uncontracted intermediate indicators
System Criteria (12)	Numerical & Code Integrity	Closure <0.05%, deterministic, <250ms solve, hybrid gate=0 off-domain

6.3 Six-Gate Demonstration & Handover Protocol (protocol.py)

Gate	Milestone Name	Required Evidence & Exit Criteria
G0	Data Readiness Review	12m historian audit, lab duplicate sigma established, criteria & envelope frozen
G1	Factory Acceptance (FAT)	Blind run on sealed withheld plant data; 17 accuracy/behaviour/fail-safe scenarios
G2	Site Acceptance (SAT)	On-prem VM + live read-only historian link; 14-day completeness, latency & lab check
G3	Performance Trial	30-90 days continuous operation across real ore excursions (<10% declared exclusions)
G4	Advisory / MPC Trial	Structured ON/OFF supervisory control trial proving variance reduction & margin lift
G5	Final Handover	Operator certification, maintenance runbook, signed Acceptance Certificate issued

6.4 Automated Report & Certificate Endpoints

Running GET `/api/v1/validation/run?plant_id=IX-OB-01&section=all&tests_pass=true` or clicking 'Run & Verdict' on the Validation page executes all 14 statistical metrics (Bias, MAE, RMSE, MAPE, R2, Nash-Sutcliffe, Theil U2, Willmott d, Bland-Altman, paired t-test, KS test, Durbin-Watson, 95% coverage) and renders a downloadable signable Markdown Acceptance Test Report (`/validation/report.md`) and Certificate (`/validation/certificate`).

Codebase Structure, 16 Operator Pages & Build-Time Warm Cache

Complete File Map of backend/ and frontend/, Industrial Historian Adapters & Precompute Pipeline

7.1 Repository Directory Map

Path / Module	Responsibility
backend/app/main.py	FastAPI app factory, CORS, GZip, X-Process-Time-ms header, background cache loader
backend/app/twin/*	psd.py, circuit.py, comminution.py, gravity.py, separation.py, dewatering.py, constraints.py
backend/app/hybrid/*	composition.py (6 modes), domain.py (3-way OOD gate), builder.py (320-row DoE trainer)
backend/app/control/*	identification.py (FOPDT + RGA), mpc.py (condensed DMC SLSQP), advisory.py
backend/app/ml/*	dataset.py, models.py (GradientBoosting), anomaly.py (Isolation/Mahalanobis + drift), registry.py
backend/app/plant3d/*	model.py (33 assets, 5 floors), geometry.py (routes + clash), bindings.py, importers.py (CAD)
backend/app/validation/*	criteria.py, metrics.py (14 stats), scenarios.py (17 tests), protocol.py (G0-G5), runner.py
backend/app/adapters/*	industrial.py: OPC UA, AVEVA PI Web API, MQTT, Kafka, SQL, CSV, REST + 168h Mock Replay
backend/scripts/precompute.py	Trains hybrid/ML/MPC/validation/replay at Docker/Render build time -> .cache/warm-*.pkl
frontend/js/main.js & api/*	Hash SPA router, 5 personas, client.js (TTL cache + retries + auto mock.js .json.gz fallback)
frontend/js/pages/* & viz/*	16 modular operator pages + Three.js plantmodel.js, scene3d.js & interactive SVG flowsheet.js
render.yaml / render.full.yaml	Render Cloud Blueprints (Static desk only vs Combined Python API + Static desk)

7.2 All 16 Operator Desk Pages & 5 Role-Based Personas

Frontend Route Group	Key Interactive Views & Capabilities
#/home · #/overview	Plant summary, 3-product contract cards, shift history, revenue/cost/margin KPIs
#/live · #/units	Interactive P&ID flowsheet, 45 live DCS tags, 8 soft sensors, 18 unit drill-downs
#/gravity · #/simulate	Washability curves, Tromp Ep partitions, full What-If simulator, Tornado sensitivity
#/optimize · #/mpc	Constrained set-point search, Monte Carlo, Pareto fronts, DMC MPC solver & DCS handover
#/ml · #/hybrid	Soft-sensor importance/PDP, drift, Gated-Residual attribution, OOD check & extrapolation
#/plant3d · #/validation	Live Three.js 3-D plant with floor slicing & CAD match; V&V runner & signable report
#/advisory · #/health · #/quality · #/admin	RGA loop pairing, mass-balance reconciliation, sensor health, API latency & persona config

How Build-Time Precompute (scripts/precompute.py) Speeds Up Cold Starts by 570x

- Training 4 hybrid models + 5 ML models + 1011 half-hourly replay nodes takes ~188 s on 1 CPU (~30 min on 0.1 CPU).
- Running `python scripts/precompute.py` during build saves SHA-256 fingerprinted `.cache/warm-<hash>.pkl` (~25 MB).
- At runtime startup, `warm_cache.load_cache()` restores everything in 0.33 seconds (<300 MB RAM)!

How to View & Run the Frontend Locally (Windows Guide)

Instant Zero-Install Python Method (Works Right Now!) + Full Live Backend Connection

8.1 Environment Check on Your PC — Why Python Works Without Node.js!

We inspected your Windows workstation: Python 3.10.11 (with FastAPI, Uvicorn, NumPy, SciPy, and scikit-learn) is ALREADY installed, whereas `node` is not currently on your system PATH. Good news: the frontend in `frontend/` is 100% pure static HTML, CSS, and native ES6 JavaScript modules — Three.js and Plotly are already bundled inside `frontend/vendor/`. Node.js is only used for optional unit tests (`jsdom`). You can serve the frontend right now using Python's built-in HTTP server!

8.2 Option A — View the Full Frontend in 5 Seconds (Offline Mock Mode)

Even without starting the backend, `frontend/data/mock/` holds compressed real API captures (`.json.gz`). With `MOCK_MODE: 'auto'` in `frontend/config.js`, the browser decompresses them natively via `DecompressionStream`:

PowerShell Commands — Instant Frontend Preview (Port 8080)

```
cd "c:\Users\svel2\Downloads\ore-beneficiation-digital-twin-v5-main\ore-beneficiation-digital-twin-v5-main\frontend"
python -m http.server 8080

# Open in Edge or Chrome:  http://localhost:8080
```

8.3 Option B — Full Live Mode (FastAPI Backend + Frontend Connected)

To run live What-If simulations, real-time constrained optimization, MPC QP solves, CAD tag matching, and live acceptance suites, open TWO PowerShell terminals side by side:

Terminal 1 — Start the FastAPI Digital Twin Backend (Port 8000)

```
cd "c:\Users\svel2\Downloads\ore-beneficiation-digital-twin-v5-main\ore-beneficiation-digital-twin-v5-main\backend"
Copy-Item .env.example .env -ErrorAction SilentlyContinue
# Optional (recommended once, ~4 mins): pre-train & cache ML/Hybrid/Replay to backend/.cache/
python scripts/precompute.py
# Launch FastAPI server:
python -m uvicorn app.main:app --reload --port 8000
# Swagger API Docs:  http://localhost:8000/docs    |   Health:  http://localhost:8000/health
```

Terminal 2 — Start the Operator Desk Frontend (Port 8080)

```
cd "c:\Users\svel2\Downloads\ore-beneficiation-digital-twin-v5-main\ore-beneficiation-digital-twin-v5-main\frontend"
python -m http.server 8080

# Open http://localhost:8080 -> Sidebar 'API status' tile turns GREEN: '● LIVE'
```

- **Zero Config Needed Locally**
  `frontend/config.js` already sets `BACKEND_BASE_URL: 'http://localhost:8000'`, and `backend/app/core/config.py` already whitelists `http://localhost:8080` and `http://127.0.0.1:8080` in `ALLOWED_ORIGINS`.
- **Switching Plants, Themes & Personas**
  Use the left sidebar controls on `http://localhost:8080` to switch between `IX-OB-01`, `IX-OB-02`, and `IX-OB-03`, toggle Dark/Light mode or Metric/Imperial units, or toggle 'Mock mode' on/off at any time.

How to Connect & Deploy to Render Cloud (Step-by-Step)

GitHub Setup, Nested Folder Warning, Backend Web Service + Frontend Static Site Blueprint

CRITICAL CHECK FIRST: Watch Out for Your Nested ZIP Folder!

Your workspace has a nested folder: ...\ore-beneficiation-digital-twin-v5-main\ore-beneficiation-digital-twin-v5-main\
You MUST run `git init` inside the INNER folder where `render.yaml`, `backend/`, and `frontend/` live!
If you push from the outer folder, Render will fail with 'requirements.txt / render.yaml not found'.
Also remove the empty stray file `frontend\asdd` before committing: Remove-Item frontend\asdd -ErrorAction SilentlyContinue

9.1 Step 1 — Push the Inner Project Folder to GitHub

PowerShell — Initialize Git & Push to Your GitHub Repository

```
cd "c:\Users\svel2\Downloads\ore-beneficiation-digital-twin-v5-main\ore-beneficiation-digital-twin-v5-main"
Remove-Item frontend\asdd -ErrorAction SilentlyContinue
git init
git add .
git commit -m "Ore Beneficiation Digital Twin v5"
git branch -M main
git remote add origin https://github.com/<YOUR_GITHUB_USER>/<YOUR_REPO_NAME>.git
git push -u origin main
```

9.2 Step 2 — Deploy Backend API (`obdt-v5-api`) on Render

In Render Dashboard (https://dashboard.render.com) -> click 'New +' -> 'Web Service' -> Connect your GitHub repo:

Render Setting	Exact Value to Enter
Root Directory	backend (MANDATORY — do not leave blank!)
Runtime / Branch	Python 3 | Branch: main | Region: Singapore (or closest)
Build Command	pip install --upgrade pip && pip install -r requirements.txt && python scripts/precompute.py
Start Command	uvicorn app.main:app --host 0.0.0.0 --port $PORT --workers 1 --timeout-keep-alive 180
Health Check Path	/health
Environment Vars	PYTHON_VERSION=3.12.3 | APP_ENV=production | MODEL_VERSION=OBDT-2026.10.5 | DEMO_MODE=true
Warm/Perf Vars	HISTORIAN_TYPE=mock | WARM_ON_START=true | WARM_HYBRID=true | OPTIMIZER_MAX_EVALS=120

Wait 5–7 minutes for `precompute.py` to build `.cache/warm-*.pkl`. Once status is LIVE, test `https://<your-api>.onrender.com/health` and copy that base URL.

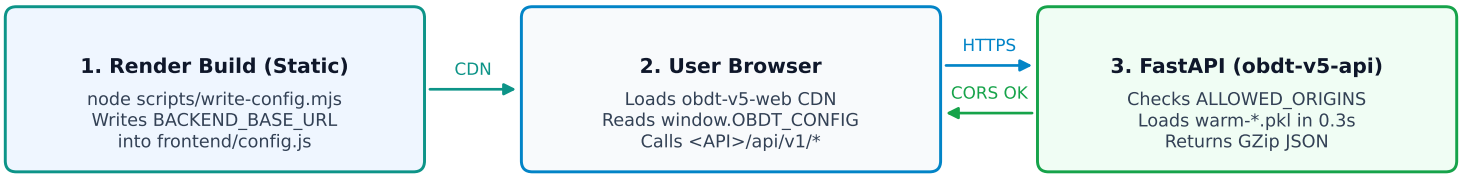
9.3 Step 3 & Step 4 — Deploy Frontend Blueprint & Link CORS

- **Step 3: Deploy Frontend (`obdt-v5-web`) via Render Blueprint**
  Render Dashboard -> 'New +' -> 'Blueprint' -> Select the same GitHub repo -> Apply. Render reads root `render.yaml` (`rootDir: frontend`, runs `node scripts/write-config.mjs` on Render's build server) and prompts for `BACKEND_BASE_URL`. Enter `https://<your-api>.onrender.com` (HTTPS, NO trailing slash, NO `/api/v1`).
- **Step 4: Close the CORS Loop on Backend (`obdt-v5-api`)**
  Copy your new static site URL (e.g., `https://obdt-v5-web.onrender.com`). Open `obdt-v5-api` -> Environment -> set `ALLOWED_ORIGINS` = `https://obdt-v5-web.onrender.com` (exact origin, no trailing slash) and Save to trigger a quick redeploy.
- **Alternative One-Step Dual Blueprint (`render.full.yaml`)**
  Want ONE Blueprint to create both services at once? Copy `render.full.yaml` over `render.yaml` before pushing (`Copy-Item render.full.yaml render.yaml -Force`), apply the Blueprint, then fill in `BACKEND_BASE_URL` and `ALLOWED_ORIGINS` on each service.

Render Cloud Verification, Troubleshooting & Tuning

How Frontend-to-Backend Connection Works on Render, CORS/Mock Diagnostics & Free Tier Tips

10.1 How Frontend & Backend Connect on Render



10.2 Fixing 'MOCK fallback, backend unreachable' (Diagnostic Table)

Open your deployed Render frontend URL -> navigate to 'Admin / config' (`#/admin`) or press F12 -> Console and run `window.OBDT_CONFIG.BACKEND_BASE_URL`:

Browser F12 / Build Symptom	Exact Cause	Fix
Prints http://localhost:8000	BACKEND_BASE_URL unset at build	Set BACKEND_BASE_URL on obdt-v5-web & redeploy
Blocked: Mixed Content error	Used http:// instead of https://	Use https://<your-api>.onrender.com
CORS Missing Allow-Origin	ALLOWED_ORIGINS missing / has '/'	Set ALLOWED_ORIGINS=https://<desk>.onrender.com
HTTP 404 /api/v1/api/v1/...	Included /api/v1 in BACKEND_BASE_URL	Use root URL only; desk appends /api/v1 itself
Pending 55s -> Mock Fallback	Free instance asleep (>15m idle)	Open <your-api>/health first to wake container
Build timeout on Free tier	Replay precompute exceeded limit	Set REPLAY_STEPS_PER_HOUR=1 & HYBRID_SAMPLES=140

10.3 Choosing the Right Render Operating Mode

Deployment Strategy	Cost	How to Configure & When to Use
1. Instant Demo (Never Sleeps)	$0 / mo	Set MOCK_MODE=true on obdt-v5-web. Instant CDN load; 3D/Hybrid/V&V work.
2. Full Live Free Tier	$0 / mo	Keep MOCK_MODE=auto. Wake <api>/health 60s ahead; warm cache loads in 0.3s.
3. Render Starter (Recommended)	$7 / mo	Always-on container (0 cold starts) + dedicated CPU for optimization & MPC.
4. Plant On-Premise Docker	Site VM	docker build backend & frontend behind firewall; set HISTORIAN_TYPE=opcua/pi.

Optional Security Hardening on Render

- Shared API Key: Set identical `API_KEY` env var on BOTH `obdt-v5-api` and `obdt-v5-web` (`X-API-Key` header).
- Password Gate on Operator Desk: Set `AUTH_MODE=password` and `APP_PASSWORD=<secret>` on `obdt-v5-web`. (`write-config.mjs` hashes `APP_PASSWORD` with SHA-256 at build time so plaintext is never stored in `config.js`.)